

Exercice 1 – Synchronisation de deux boucles

Afficher les nombres de 1 à 10 sur une seule ligne à l'aide :

1. d'une boucle `while`
 2. d'une boucle `for`
-

Exercice 2 – Parcours conditionnel inverse

À partir de la liste suivante :

```
numbers = [4, 7, 1, 9, 3, 6, 2]
```

Afficher les éléments **dans l'ordre inverse**, mais uniquement ceux qui sont impairs.

Exercice 3 – Accès indexé contrôlé

Étant donné :

```
s = "infrastructure"
```

Afficher un caractère sur deux à l'aide :

1. d'une boucle avec index
2. du slicing

Exercice 4 – Détection de motif dans une chaîne

À partir de la chaîne suivante :

```
text = "programmationpython"
```

Afficher toutes les positions où la lettre 'p' apparaît.

Contrainte : - utiliser une boucle avec index

Exercice 5 – Reconstruction de chaîne

À partir de :

```
s = "abcdefgh"
```

Créer une nouvelle chaîne contenant uniquement les caractères aux index pairs.

Interdiction : - ne pas utiliser de slicing

Exercice 6 – Comparaison chaîne / liste

Étant donné :

```
s = "python"
```

1. Transformer la chaîne en liste de caractères
2. Parcourir cette liste et afficher chaque caractère en majuscule
3. Recréer une chaîne à partir du résultat

Exercice 7 – Découpage logique d'une liste

À partir de la liste suivante :

```
values = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Créer trois listes :

- la première moitié
- la seconde moitié
- les 3 éléments centraux uniquement

Exercice 8 – Boucle `while` avec index explicite

Parcourir la liste suivante à l'aide d'une boucle `while` :

```
data = [10, 20, 30, 40, 50]
```

Afficher chaque élément ainsi que son index.

Exercice 9 – Analyse de symétrie

Étant donné :

```
s = "radar"
```

Déterminer si la chaîne est un palindrome (mot se lisant des deux sens).

Exercice 10 – Reconstruction conditionnelle d'une liste

À partir de :

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Créer une nouvelle liste contenant :

- les nombres pairs multipliés par 2
- les nombres impairs inchangés